



Manual técnico y guía de instalación

Laboratorio de Sistemas Operativos 1 – 2S2025

Carnet: Ingrid Lorena Garcia Yantuche

Nombre: 201700399

Proyecto 1

Descripción

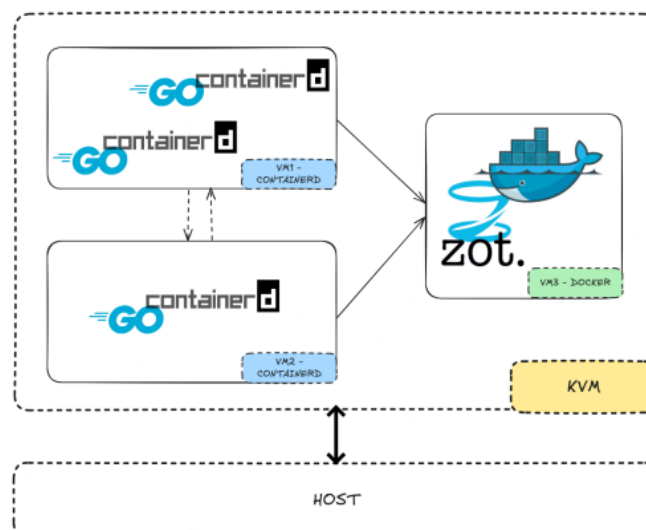
El presente proyecto tiene como objetivo principal el diseño e implementación de un entorno virtualizado que integre el uso de máquinas virtuales (VMs) y contenedores, empleando tecnologías modernas como Docker, Containerd, Go y Zot.

La arquitectura implementada permite simular entornos reales de desarrollo utilizados en la industria, enfocados en la contenerización, el almacenamiento de imágenes de contenedores y la gestión eficiente de recursos.

El proyecto fue desarrollado en cuatro fases siguiendo la metodología ágil SCRUM:

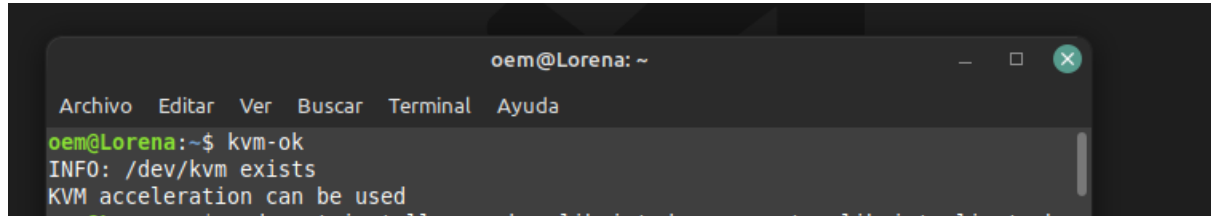
1. Preparación del entorno virtualizado (KVM + 3 VMs).
2. Desarrollo de APIs REST en Go y contenerización con Docker/Containerd.
3. Configuración de registro privado de imágenes con Zot y distribución de imágenes.
4. Validación de la comunicación entre servicios, documentación y entrega final.

Diagrama de arquitectura

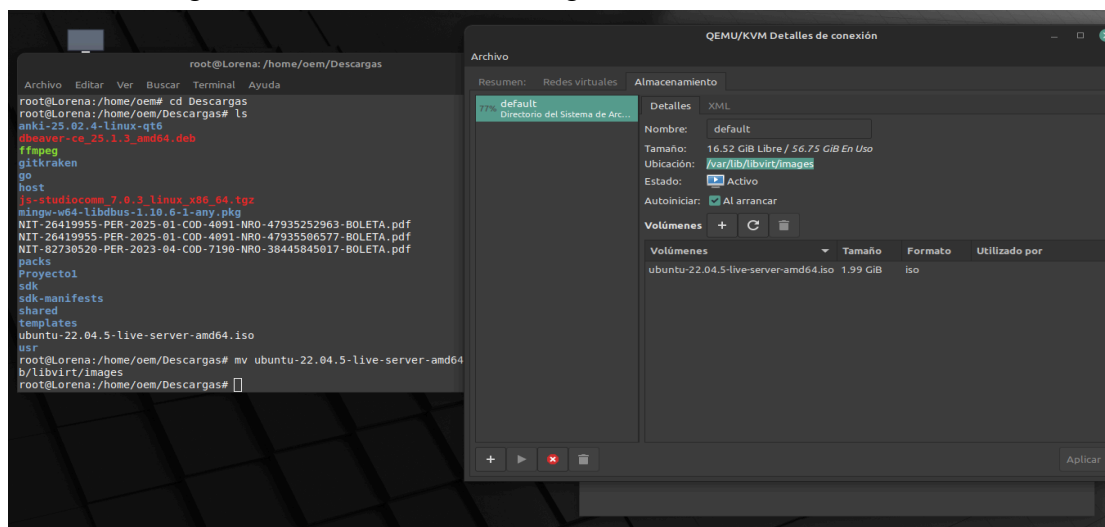


Verificación de KVM

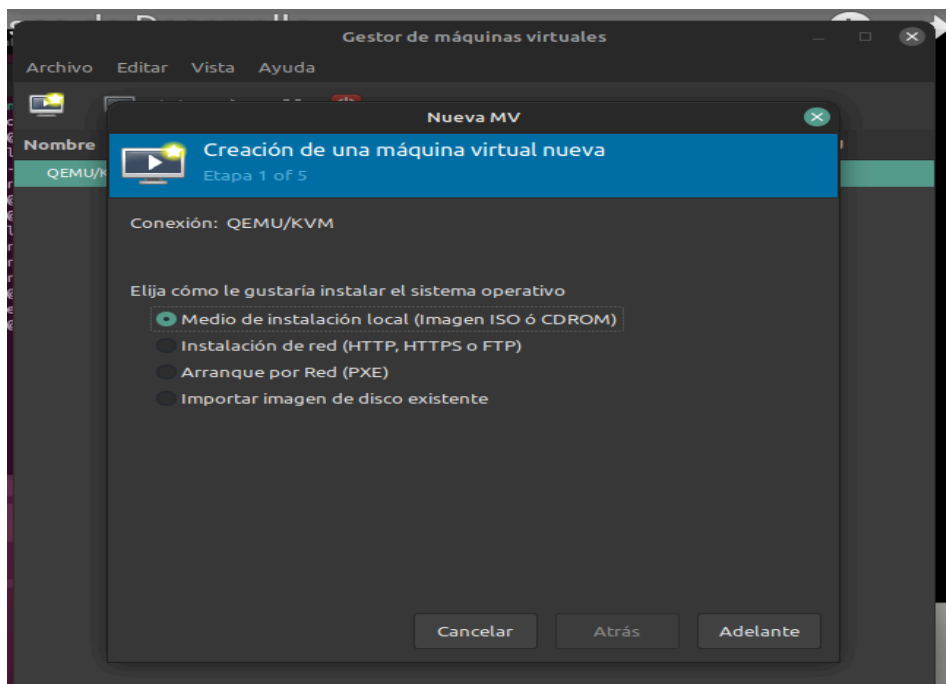
Es una tecnología de virtualización incluida en el kernel de Linux que permite convertir a un sistema Linux en un hipervisor (es decir, en una máquina capaz de ejecutar otras máquinas virtuales).

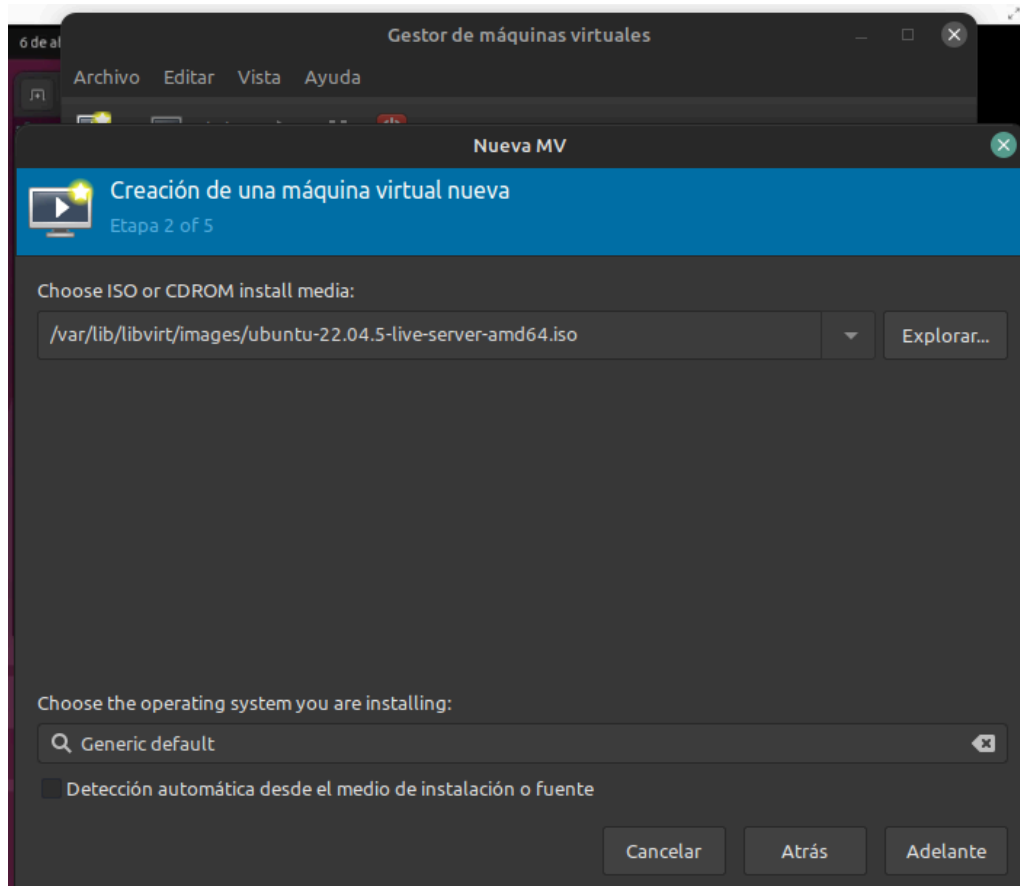
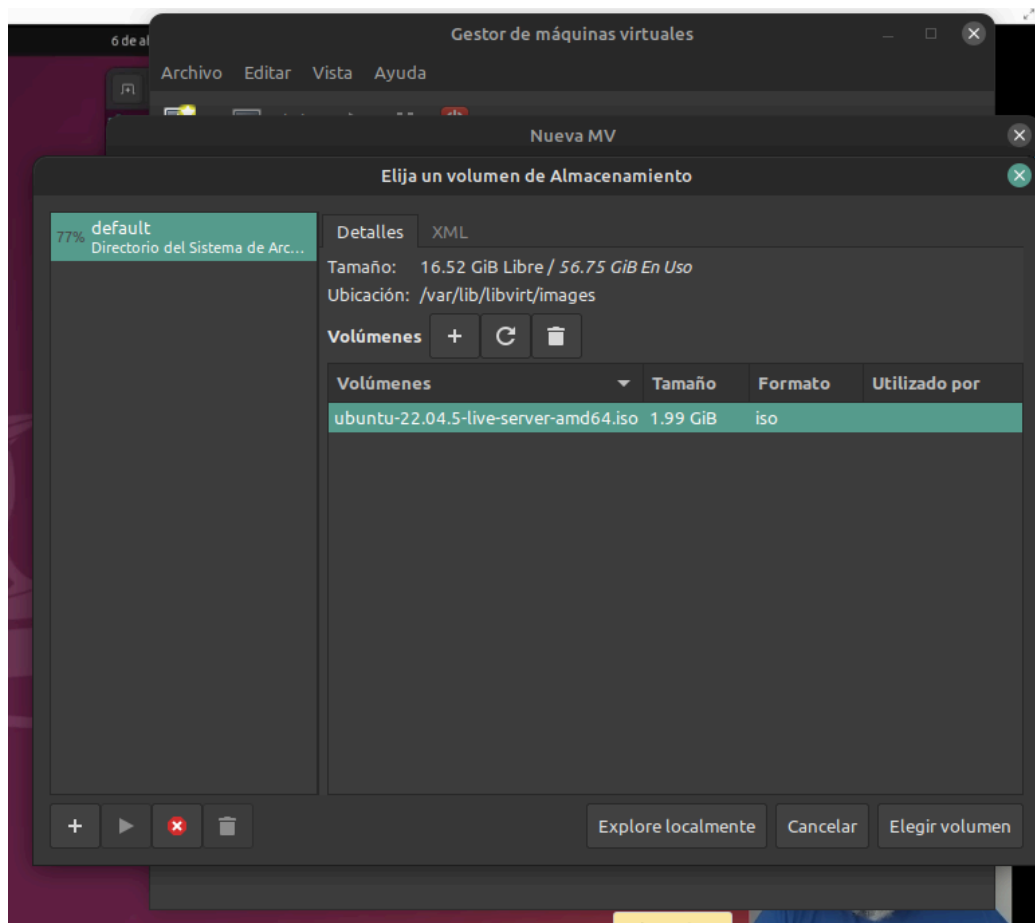


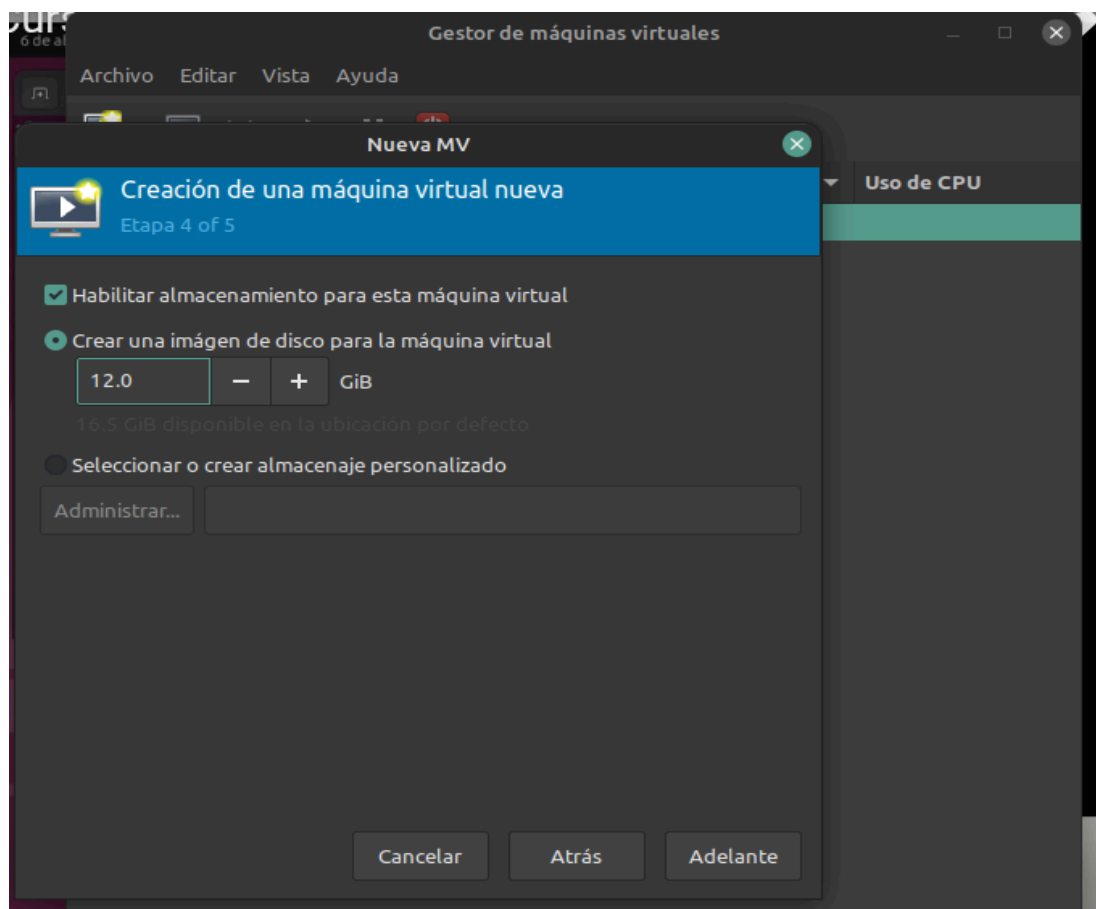
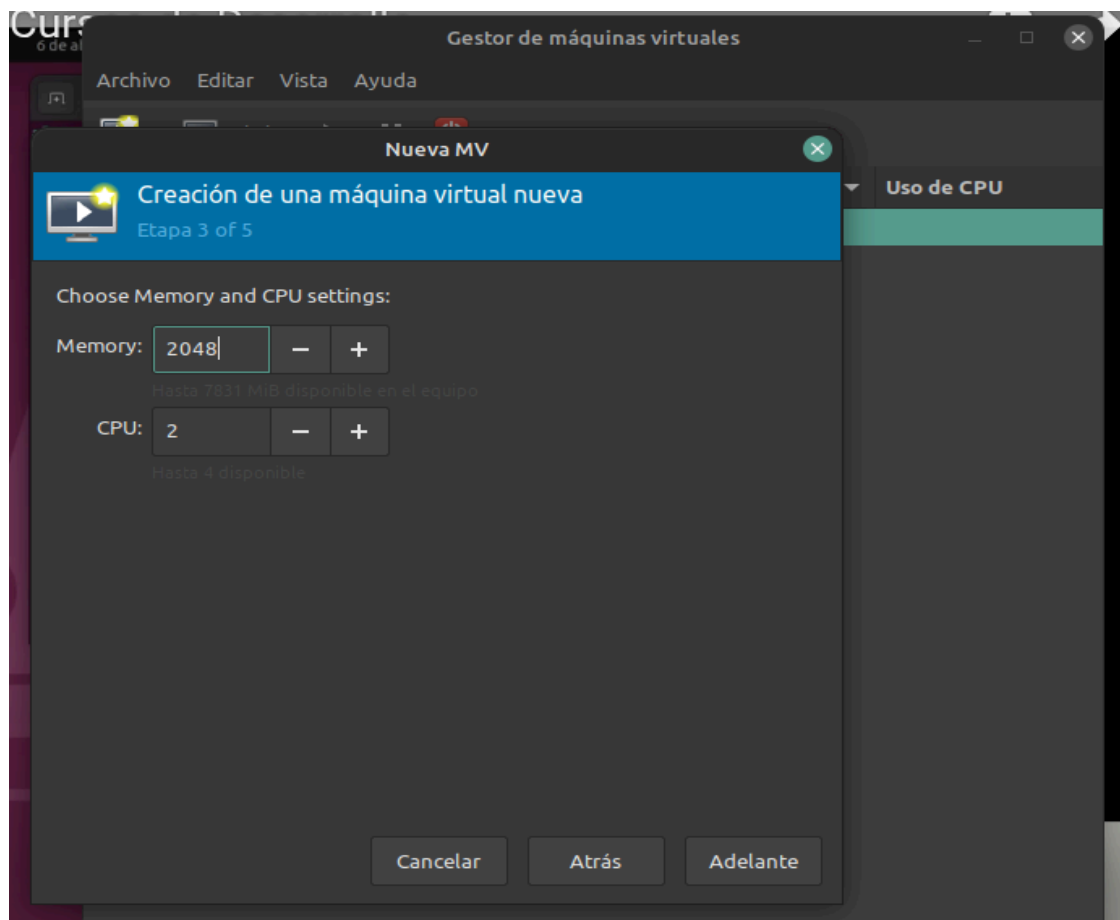
Se movio la imagen iso a /var/lib/libvirt/images

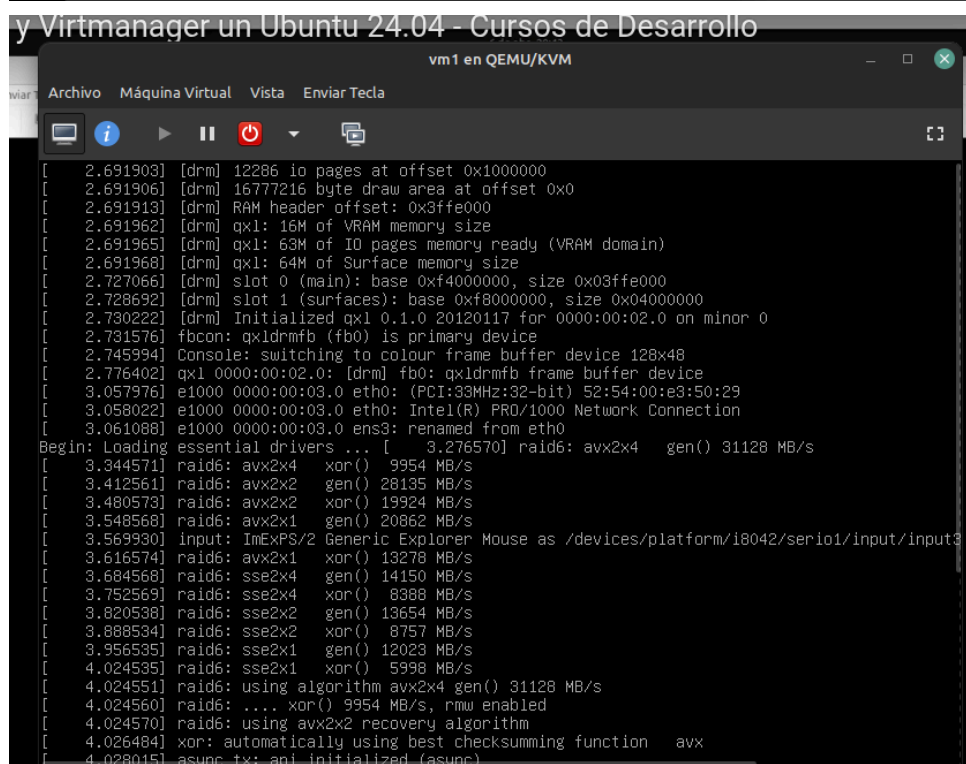
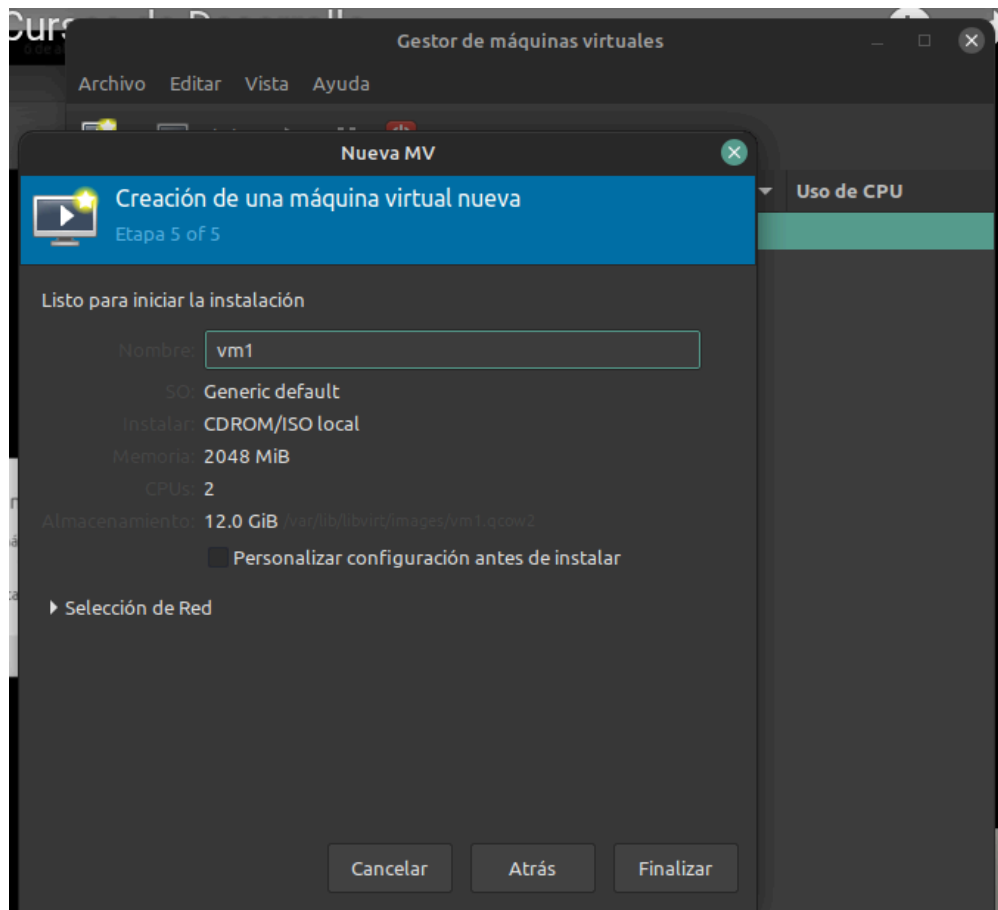


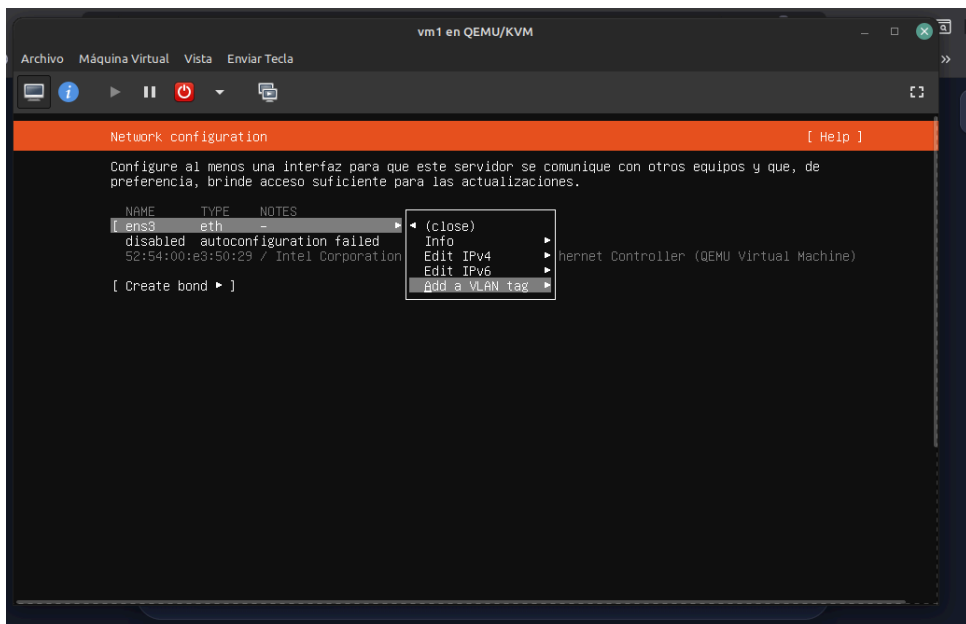
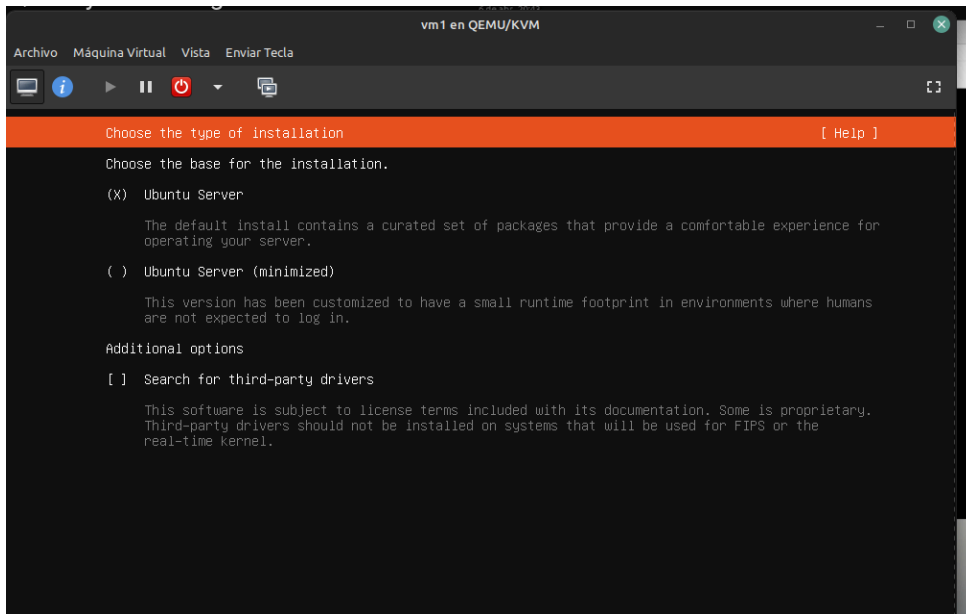
Crear una máquina virtual nueva

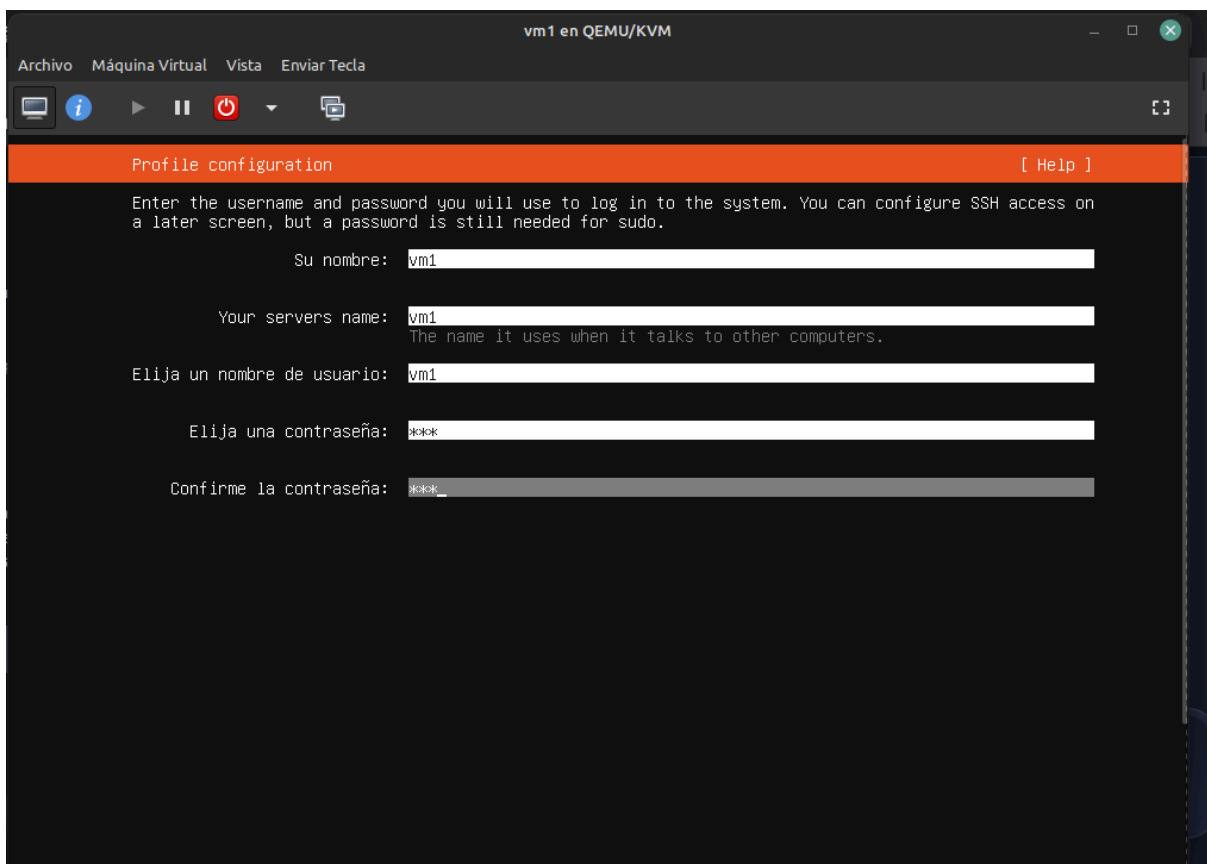
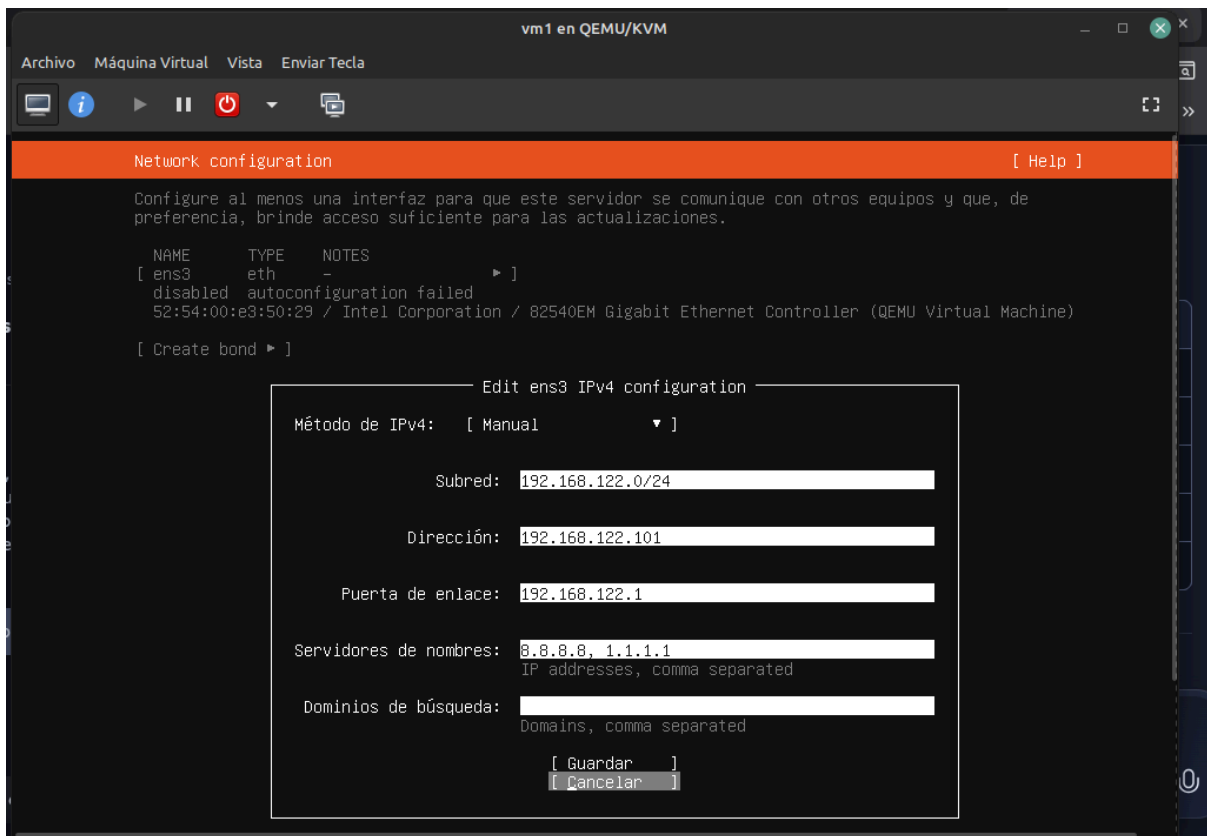












Información General de la máquinas virtuales

VM	Uso	RAM	CPU	Disco	Notas
VM3	API1 + API2 (containerd)	1.5 GB	1	8 GB	APIs 1, 2 y containerd
VM2	API3 (containerd)	1 GB (temporal 2 GB)	1	8 GB	Solo una API, más ligera
VM1	ZOT (docker)	4 GB (temporal 2 GB)	2	12 GB	Registro privado de imágenes

Crea VM1 (Zot con Docker)

Virt-Manager:

- 192.168.122.134

Instala Docker y levanta Zot:

```
sudo apt update && sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo docker run -d --name zot -p 5000:5000 zotregistry/zot
```

Crea VM2 (API3 con containerd)

- IP: 192.168.122.102

Instalación de Containerd:

```
sudo apt update && sudo apt install -y containerd
sudo mkdir -p /etc/containerd
sudo containerd config default | sudo tee
/etc/containerd/config.toml >/dev/null
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/'
/etc/containerd/config.toml
sudo systemctl enable --now containerd
sudo ctr images pull docker.io/library/alpine:3.19
sudo ctr run --rm -t docker.io/library/alpine:3.19 test sh -c 'echo
"VM2 OK" '
```

Crea VM3 (API1, API2 con containerd)

Virt-Manager:

- 192.168.122.103

Instalación de Containerd:

```
sudo apt update && sudo apt install -y containerd
sudo mkdir -p /etc/containerd
sudo containerd config default | sudo tee
/etc/containerd/config.toml >/dev/null
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/'
/etc/containerd/config.toml
sudo systemctl enable --now containerd
sudo ctr images pull docker.io/library/alpine:3.19
sudo ctr run --rm -t docker.io/library/alpine:3.19 test sh -c 'echo
"VM2 OK" '
```

Api

Tecnologías

- Go, fiber

```
func main() {
    app.Get("/", func(c *fiber.Ctx) error {
        return c.JSON(fiber.Map{
            "mensaje": fmt.Sprintf(
                "io/ioutil"
                "net/http"

                "github.com/gofiber/fiber/v2"
            )
        })
    })

    func main() {
        carnet := "201700399"
        app := fiber.New()

        // Endpoint raíz, devuelve JSON
        app.Get("/", func(c *fiber.Ctx) error {
            return c.JSON(fiber.Map{
                "mensaje": fmt.Sprintf(
                    "Hola, responde la API: API1 en la VM3, desarrollada por el estudiante Ingr
                    carnet,
                ),
            })
        })

        // Llamada a API2
        app.Get("/api1/"+carnet+"/llamar-api2", func(c *fiber.Ctx) error {
            resp, err := http.Get("http://vm3:8082/") // Endpoint raíz de API2
            if err != nil {
```

- dockerfile

```
# ----- Etapa de compilación -----
FROM golang:1.22-alpine AS builder

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiar archivos de dependencias
COPY go.mod go.sum ./
RUN go mod download

# Copiar el resto del código fuente
COPY . .

# Compilar la aplicación (se genera un binario llamado apil)
RUN CGO_ENABLED=0 GOOS=linux go build -o apil main.go

# ----- Etapa final (imagen ligera) -----
FROM alpine:3.22

# Directorio de trabajo
WORKDIR /app

# Copiar el binario compilado desde la etapa anterior
COPY --from=builder /app/apil .

# Exponer el puerto en el que corre tu API
EXPOSE 8081
```

VM IP	API	Endpoint 1	Endpoint 2
192.168.122.103	Api1	GET /api1/201700399/llamar-api2	GET /api1/201700399/llamar-api3
192.168.122.103	Api2	GET /api2/201700399/llamar-api1	GET /api2/201700399/llamar-api3
192.168.122.102	Api3	GET /api3/201700399/llamar-api1	GET /api3/201700399/llamar-api2

curl http://vm3:8081/api1/201700399/llamar-api1

En Host

Construir imagen - esto se hace para las 3 APIS

docker build -t api1-vm3:liviana .

docker build -t api2-vm3:liviana .

docker build -t api3-vm2:liviana .

Correr la imagen

docker run -d -p 8081:8081 api1-vm3:liviana

docker run -d -p 8082:8082 api2-vm3:liviana

docker run -d -p 8083:8083 api3-vm2:liviana

Subir imagen desde host a la vm que tiene zot

- primero verificar que tenga esto
sudo nano /etc/docker/daemon.json

docker tag api1-vm3:liviana 192.168.122.134:5000/api1-vm3:liviana

docker push 192.168.122.134:5000/api1-vm3:liviana

Repetir lo anterior para las demás imágenes

docker tag api2-vm3:liviana 192.168.122.134:5000/api2-vm3:liviana

docker push 192.168.122.134:5000/api2-vm3:liviana

docker tag api3-vm2:liviana 192.168.122.134:5000/api3-vm2:liviana

docker push 192.168.122.134:5000/api3-vm2:liviana

Prueba para verificar que se subió correctamente:

curl http://localhost:5000/v2/_catalog

Esto se hace desde las máquinas que tienen containerd y ctr

En máquina virtual vm3

API1

```
sudo ctr images pull --plain-http vm1:5000/api1-vm3:liviana
sudo ctr run -d --net-host vm1:5000/api1-vm3:liviana my-api1-vm3
```

API2

```
sudo ctr images pull --plain-http vm1:5000/api2-vm3:liviana
sudo ctr run -d --net-host vm1:5000/api2-vm3:liviana my-api2-vm3
```

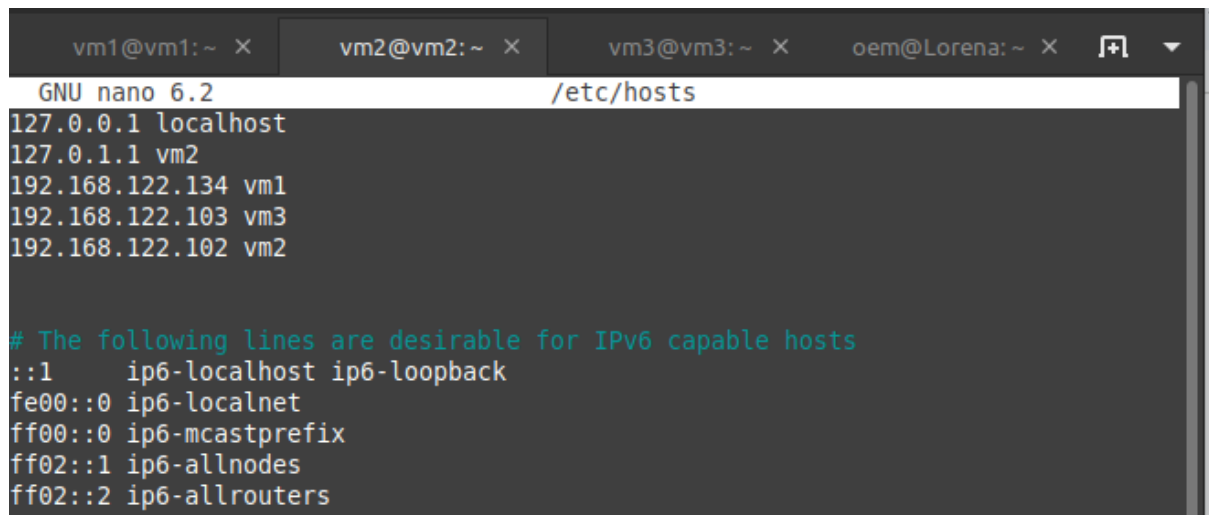
En máquina virtual vm2

API3

```
sudo ctr images pull --plain-http vm1:5000/api3-vm2:liviana
sudo ctr run -d --net-host vm1:5000/api3-vm2:liviana my-api3-vm2
```

Asignar nombre a las Vm

sudo nano /etc/hosts

A screenshot of a terminal window with four tabs: 'vm1@vm1:~', 'vm2@vm2:~', 'vm3@vm3:~', and 'oem@Lorena:~'. The active tab is 'vm2@vm2:~'. The terminal shows the 'GNU nano 6.2' editor editing the '/etc/hosts' file. The content of the file is as follows:

```
127.0.0.1 localhost
127.0.1.1 vm2
192.168.122.134 vm1
192.168.122.103 vm3
192.168.122.102 vm2

# The following lines are desirable for IPv6 capable hosts
::1      ip6-localhost ip6-loopback
fe00::0  ip6-localnet
ff00::0  ip6-mcastprefix
ff02::1  ip6-allnodes
ff02::2  ip6-allrouters
```

VER IP DE LAS VM

sudo virsh domifaddr vm1

Ver todas las máquinas virtuales

sudo virsh list --all

Verificaciones cruzadas

ping -c 3 192.168.122.10

```
vm1@vm1:~$ ping -c 3 192.168.122.102
PING 192.168.122.102 (192.168.122.102) 56(84) bytes of data.
64 bytes from 192.168.122.102: icmp_seq=1 ttl=64 time=0.533 ms
64 bytes from 192.168.122.102: icmp_seq=2 ttl=64 time=1.82 ms
64 bytes from 192.168.122.102: icmp_seq=3 ttl=64 time=1.46 ms

--- 192.168.122.102 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2025ms
rtt min/avg/max/mdev = 0.533/1.272/1.821/0.542 ms
vm1@vm1:~$ ping -c 3 192.168.122.103
PING 192.168.122.103 (192.168.122.103) 56(84) bytes of data.
64 bytes from 192.168.122.103: icmp_seq=1 ttl=64 time=4.05 ms
64 bytes from 192.168.122.103: icmp_seq=2 ttl=64 time=2.16 ms
64 bytes from 192.168.122.103: icmp_seq=3 ttl=64 time=1.88 ms

--- 192.168.122.103 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 1.875/2.693/4.048/0.964 ms
vm1@vm1:~$
```

Inicializar vm2 y vm3

sudo ctr tasks start my-api1-vm3

sudo ctr tasks start my-api2-vm3

```
oem@Lorena: ~
Archivo  Editar  Ver  Buscar  Terminal  Pestañas  Ayuda

oem@Lorena: ~  x  oem@Lorena: ~  x  oem@Lorena: ~  x  [icon]  v

vm3@vm3:~$ sudo ctr tasks start my-api1-vm3
sudo ctr tasks start my-api2-vm3
API1 corriendo en puerto 8081...

Fiber v2.52.9
http://127.0.0.1:8081
(bound on host 0.0.0.0 and port 8081)

Handlers ..... 6  Processes ..... 1
Prefork ..... Disabled  PID ..... 1

^API2 corriendo en puerto 8082...

Fiber v2.52.9
http://127.0.0.1:8082
(bound on host 0.0.0.0 and port 8082)

Handlers ..... 6  Processes ..... 1
Prefork ..... Disabled  PID ..... 1
```

sudo ctr tasks start my-api3-vm2

```
vm1@vm1:~$ curl http://vm2:8083/api3/201700399/llamar-api2
{"mensaje":"Hola, responde la API: API2 en la VM3, desarrollada por el estudiante Ingrid Lorena G
vm1@vm1:~$ curl http://vm2:8083/api3/201700399/llamar-api2
{"mensaje":"Hola, responde la API: API2 en la VM3, desarrollada por el estudiante Ingrid Lorena G
arcia Yantuche con carnet: 201700399"}vm1@vm1:~$
```